@prism project

Christophe BAL

1 Jan 2026 – Version 1.3.0

The `@prism` project[1] provides color palettes that can be used to create content based on expressive colors.

Last changes

1.3.0
2026-01-01

Break.

- `luadraw` product becomes `Lua` product.
- `Lua` product: the dict-like variable `palNames` has been removed. Therefore, the responsibility of adding this specific variable will lie with `luadraw`.

New.

- New `CSS` and LaTeX products have been coded.
- The `Palettable` palettes have been added.
- Three semantic palettes `TdocBW`, `TdocCol` and `TdocDark` have been created to be used by the `tutodoc` class (LaTeX creation process used).
- Added an `online demo of @prism` (the `CSS` product showcase is used).

Update.

- Manual.
  - Added lists of renamed and removed palettes.
  - Added two appendixes, one with all palettes grouped by user-friendly categories, and another showing similar palettes.
  - Added explanations about the utility of palettes for non-designers.
  - Added explanations about the minimal common optional sets of extra features implemented.
- Better palette references.
- Better showcases.
  - Several category `PDF` files are proposed (one `PDF` per category and theme).
  - 2D chart and Delaunay triangulation added.

Fix.

- `Lua` product: the `use-std.tex` and `use-dark.tex` codes have been fixed.

[1] The name comes from "***@·esthetic P·roducts*** *for* ***R·epresenting I·nformative S·cientific M·aps***". This name is a double play on words: [1] a prism splits light into an informative spectrum, symbolizing how data are decomposed into meaningful color, and [2] "@" read as "at" indicates where the light meets the prism to be broken down into an informative spectrum.

Contents

I. Motivations **3**

II. Where do the color palettes come from? **4**

III. Reuse from... **5**

IV. How to choose a palette? **6**

V. Supported implementations **6**
1. JSON, the versatile default format 6
2. CSS 7
3. LaTeX 8
   a. Basic use 8
   b. Creating palettes from scratch 8
   c. Creating palettes from existing ones 9
   d. Retrieving the internal definition of a palette 9
4. Lua 9
   a. Basic use 10
   b. Creating palettes from existing ones 10

VI. Contribute via Git **10**
1. Complete the translations 10
   a. The en folder 11
   b. The changes folder 11
   c. The status folder 11
   d. The README.md and LICENCE.txt files 11
   e. New translations 11
2. Improving the source code 11

VII. History **12**

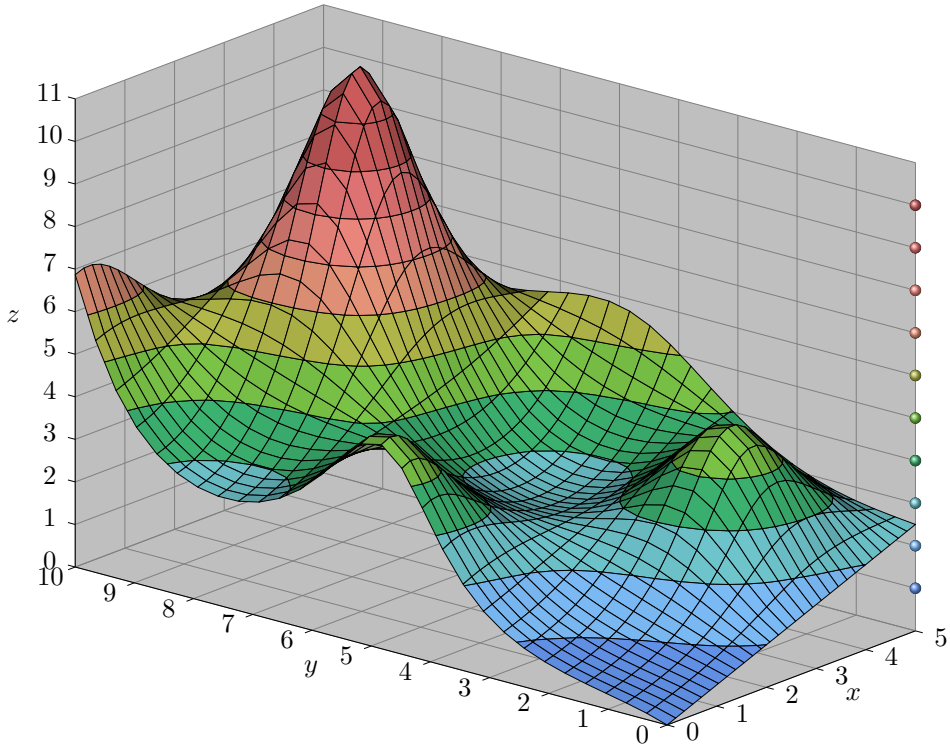
Appendix 1 – All palette names **14**

Appendix 2 – Palettes by category **17**
- Colorblind palettes 17
- Cyclic palettes 19
- Dark palettes 20
- Divergent palettes 21
- Qualitative palettes 23
- Semantic palettes 27
- Sequential palettes 28

I. Motivations

This project was born out of a desire to enhance `luadraw` with a set of color palettes to easily produce graphics like the 3D plot shown below. Despite the availability of palette generation tools such as `Coolors`, the choice has been made to offer pre-built palettes, because this serves several key purposes for non-designers.

- **Reduced cognitive load:** pre-configuration eliminates design decisions, allowing to focus on content.
- **Consistency:** it becomes easy to ensure visual coherence across documents and projects, regardless of technology.
- **Accessibility compliance:** some pre-configured palettes guarantee colorblind-safe combinations.
- **Perceptual uniformity:** most palettes ensure distinguishable and meaningful colors for data visualization, while others serve artistic purposes.

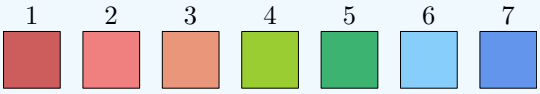


Technically, to generate the graphics in this section, `luadraw` is provided with a finite list of colors and uses linear interpolations to calculate the intermediate color values. In the example above, the finite color palette used is defined as follows.

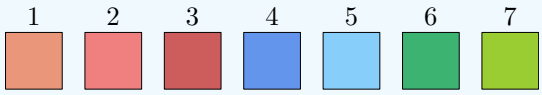
Using this palette, `luadraw` can produce the following spectrum, allowing us to create the graphic above.

Note.

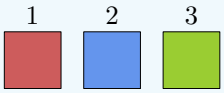
Using the `Lua` *implementation of* `@prism`*, see the section V-4, via* `luadraw`*, we can create the palettes below made from the previous one named* `'GeoRainbow'`*. Each instruction used is given below each palette.*




```
getPal('GeoRainbow', {reverse = true})
```

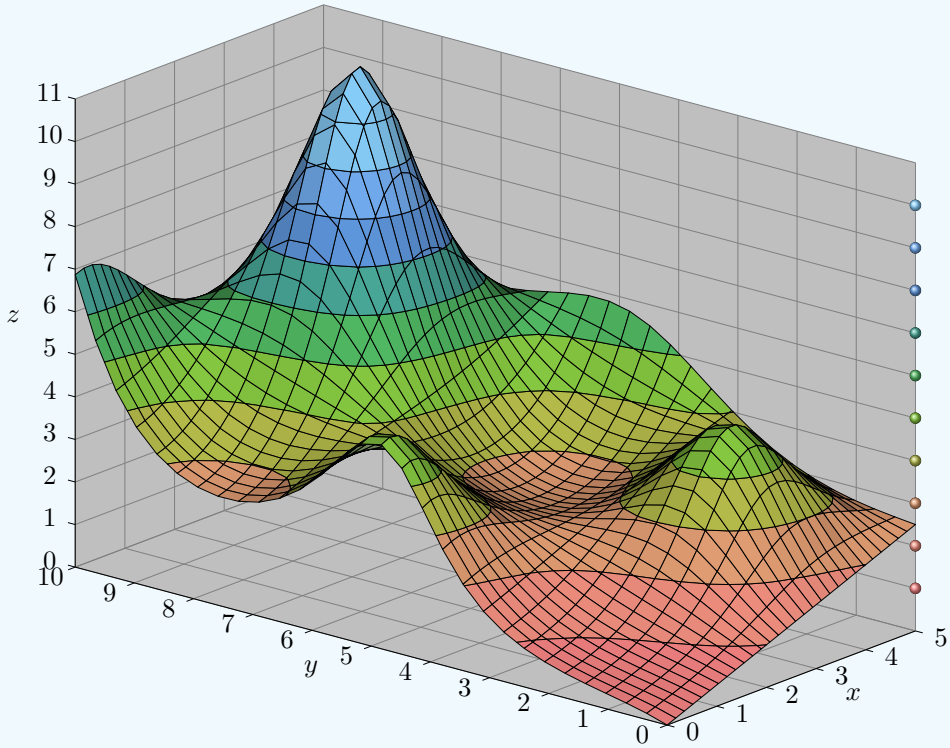


`getPal('GeoRainbow', {shift = 3})`



`getPal('GeoRainbow', {extract = {7, 1, 4}})`

This features provide remarkable creative flexibility: with the same surface as before, using the setting `getPal('GeoRainbow', {extract = {6, 5, 4, 3, 1, 2}})` *instead of* `getPal('GeoRainbow')`*, we change the visual tone, shifting from a seaside feel to a snow-covered world.*



II. Where do the color palettes come from?

`@prism` includes some original creations, but most color palettes are derived from the projects listed below.

- `Asymptote` is used, but currently offers nothing beyond `Matplotlib` (despite different implementations).
- `CarbonPlan` provides palettes designed for data visualization..
- `CartoColor` palettes are extracted from `Palettable` project.
- `cmocean` palettes are extracted from `Palettable` project.
- `Colorbrewer` provides professional color palettes for mapping and data visualization.
- `Light and Bartlein` palettes are extracted from `Palettable` project.
- `Matplotlib` compiles color maps from diverse projects.
- `MyCarta` palettes are extracted from `Palettable` project.
- `Plotly` palettes are extracted from `Palettable` project.
- `Scientific Coulour Maps` provides palettes designed for colorblind accessibility.
- `Tableau` palettes are extracted from `Palettable` project.
- `Wes Anderson Palettes` palettes are extracted from `Palettable` project.

We retain only palettes that comply with the following rules.

- **No repetition.** Unlike Matplotlib,[2] `@prism` use a one-to-one map from names to palettes.
- **No reversed versions.** Unlike Matplotlib,[3] `@prism` never includes reversed palettes as fixed data.

Note.

Adding new palettes to `@prism` is straightforward (no coding skills required). See section VI-2 to get started.

III. Reuse from...

Here are the key points to remember when using palettes similar to those offered by projects listed in the section II.

1. `@prism` uses strict alphanumeric `CamelCase` naming. Therefore, palette names such as `berlin` and `gist_heat` become `Berlin` and `GistHeat` respectively. This implies that palette names from different projects are compared internally using lowercase. Appendix 1 (page 14) lists all palette names to help you find the correct `CamelCase` spelling.
2. Matplotlib palettes with a name ending with `_r` (reversed color order) are not included in `@prism`.
3. The following palettes have been renamed: $\boxed{\Rightarrow}$ indicates a name modification, with the `@prism` name displayed on the right.

CARTOColors	Prism	⇒	PrismCC
CarbonPlan	Blues	⇒	BluesCP
	Cool	⇒	CoolCP
	Earth	⇒	EarthCP
	Greens	⇒	GreensCP
	Greys	⇒	GreysCP
	Oranges	⇒	OrangesCP
	Purples	⇒	PurplesCP
	Rainbow	⇒	RainbowCP
	Reds	⇒	RedsCP
Colorbrewer	YlOrRd	⇒	YlOrRdCBW
Plotly	Bluered	⇒	BlueRedPLY
	Hot	⇒	HotPLY
	Jet	⇒	JetPLY
	Rainbow	⇒	RainbowPLY
Tableau	Gray	⇒	MidGray

4. The following palettes are excluded because they duplicate `@prism` palettes either directly or in reversed order, except that exact duplicates (same name and colors) are omitted: we use $\boxed{=}$ for equality, and $\boxed{\rightleftharpoons}$ for reversal, and the rightmost palettes are the ones retained in `@prism`.

Matplotlib	Gray	⇌	Binary
	GistYarg	⇌	GistGray
Plotly	D3	=	Tab10
Tableau	Tableau	=	Tab20

Caution.

Most `@prism` implementations add the `pal` prefix to standardized `CamelCase` names. See the section V.

[2] Some Matplotlib palettes are duplicated, likely for historical reasons.
[3] Most Matplotlib color maps have a reversed version named with the `_r` suffix, possibly for performance reasons.

Note.

Most `@prism` implementations provide methods to easily obtain reversed palettes, sub-palettes, and color-shifted palettes. See the section V.

IV. How to choose a palette?

Here are a few tips to help you select your color palette.

1. An `online demo of @prism`[4] proposes an intuitive way to look for a palettes.
2. Appendix 2 (page **??**) presents all palettes, organized by category, along with a visualization of their continuous spectra, or their corresponding discrete palette definitions.

V. Supported implementations

All implementations are located in the `products` folder. Each implementation provides the following features:

- Palette formats in both modular (one file per palette) and monolithic (all palettes in a single file) versions.
- Palette definitions in both high-fidelity (original size) and small (currently 40 colors) size.
- Select specific colors from an existing palette using their indices.
- Shift the palette left (negative value) or right (positive value) by any number of steps.
- Reverse the order of the colors.

Important.

To explain how new palettes can be built, we will refer to the colors in the standard palette as `coul_1`, `coul_2`, etc., and suppose that the extracted indices are `{1, 3, 6, 9}`, the shift used is 1+, and the `reverse` option is enabled. The new palette will then be built sequentially as follows: first `{coul_1, coul_3, coul_6, coul_9}` (extraction), second `{coul_9, coul_1, coul_3, coul_6}` (shift to the right), and finally `{coul_6, coul_3, coul_1, coul_9}` (reverse).

Note.

Extra features are limited to discrete palette operations. For example, color interpolation is not provided, as this is usually handled out of the box by visualization and formatting tools.

1. JSON, the versatile default format

The `JSON` product allows unsupported programming languages to integrate `@prism` palettes easily. Below are the first lines of the `palettes-hf.json` file.

```
{
  "Accent": [
    [0.49803922, 0.78823529, 0.49803922],
    [0.74509804, 0.68235294, 0.83137255],
    [0.99215686, 0.75294118, 0.5254902],
    [1, 1, 0.6],
    [0.21960784, 0.42352941, 0.69019608],
    [0.94117647, 0.00784314, 0.49803922],
    [0.74901961, 0.35686275, 0.09019608],
    [0.4, 0.4, 0.4]
  ],
  ...
}
```

[4] https://projetmbc.github.io/for-writing/

2. CSS

Note.

The `HTML`, `CSS`, and `JavaScript` files for product development and demonstration were created using `Claude` and `Gemini` AI assistants.

Note.

All formats are provided: modular (each palette is in a dedicated file) and monolithic (files provide all the palettes).

Each palette color is defined as an individual variable named according to the pattern `--pal<name>-<nb>`, where `<name>` is the standard palette name and `<nb>` is the desired index ranging. Each palette color variable is defined as an `RGB` value using percentage notation. For example, the file `palettes-hf/GistHeat.css` looks like the following partial code.

```
:root {
  --palGistHeat-1: rgb(0% 0% 0%);
  --palGistHeat-2: rgb(0.59% 0% 0%);
  /* Other RBG colors.*/
  --palGistHeat-255: rgb(100% 99.22% 98.43%);
  --palGistHeat-256: rgb(100% 100% 100%);
}
```

Here is one first possible use case.

```
.warning-text {
  color: var(--palGistHeat-3);
}

.gist-heat-gradient {
  background: linear-gradient(
    90deg,
    var(--palGistHeat-1),
    var(--palGistHeat-64),
    var(--palGistHeat-128),
    var(--palGistHeat-256)
  );
}
```

The example below demonstrates creating gradient variables through selective color extraction and custom reordering.

```
:root {
  --transformed-gist-heat-gradient: linear-gradient(
    90deg,
    var(--palGistHeat-6),
    var(--palGistHeat-3),
    var(--palGistHeat-9),
    var(--palGistHeat-1)
  );
}

.transformed-gist-heat-gradient {
  background: var(--transformed-gist-heat-gradient);
}
```

3. LaTeX

a. Basic use

Note.

Just one kind of format is provided: modular (each palette is in a dedicated file).

Accessing a single palette color is straightforward: use `\palUse{<name>}{<index>}` where `<name>` is the standard palette name (without prefix), and `<index>` is the color number (ranging from 1 to 10). For example, `\palUse{GistHeat}{8}` is the eighth color of the `GistHeat` palette, an `xcolor` format color that can be easily used as shown in the following compilable example.

```
\documentclass{article}

% Load the palette (LaTeX can't work with all the palettes).
\usepackage{palettes-hf/GistHeat}

% Load the palette API.
\usepackage{palapi}

\usepackage{tikz}

\begin{document}

\textcolor{\palUse{GistHeat}{8}}{\bfseries Colored text.}

Representation of the first ten palette colors.

\begin{tikzpicture}
  \foreach \i in {1,...,10} {
    \fill[\palUse{GistHeat}{\i}]
      (1.25*\i - 1, 0) rectangle (1.25*\i, 1);
  }
\end{tikzpicture}

\end{document}
```

b. Creating palettes from scratch

For creating new palettes manually, the following high-level commands are available.

1. `\palCreateFromNames` works with a comma separated list of named colors, while `\palCreateFromRGB` creates a palette by entering it as an array-like variable.
2. `\palSize{<name>}` returns the palette size (useful for loops, for example).

The following example demonstrates the `\palCreateFromRGB` and `\palCreateFromNames` commands.

```
\usepackage{palapi}

\usepackage[svgnames]{xcolor}

\palCreateFromRGB{MyRGBPal}{
  {0.0, 0.0, 0.0},
  {0.4, 0.0, 0.2},
  {0.8, 0.2, 0.0},
}

\palCreateFromNames{MyNameUsePal}{
  YellowGreen,
  green!60!black,
  LimeGreen!80,
}
```

Note.

All built-in palettes are created using the `\palCreateFromRGB` *macro.*

A lower-level approach is also available through the following commands.

1. `\palNew{<name>}` initializes a new (empty) palette.
2. `\palAddNames{<name>}{<color-using-names>}` appends a color defined with named colors to the palette.
3. `\palAddRGB{<name>}{<r>, <g>, <b>}` appends an `RGB` color to the palette, where `<r>`, `<g>`, and `<b>` are decimal values ranging from 0 to 1.

The following example demonstrates the flexibility offered by these low-level commands.

```
\usepackage{palapi}

\usepackage[svgnames]{xcolor}

\palNew{LowLevelPal}
\palAddNames{LowLevelPal}{IndianRed}
\palAddRGB{LowLevelPal}{0.0, 0.0, 0.0}
\palAddNames{LowLevelPal}{green!60!black}
\palAddRGB{LowLevelPal}{0.8, 0.2, 0.0}
```

c. Creating palettes from existing ones

Building new palettes by transforming existing ones can be achieved using the `\palCreateFromPal` command, which has the signature `\palCreateFromPal{<new-name>}[<options>]{<existing-name>}`. The following example shows how to do this (all options are used).

```
\usepackage{palettes-hf/Blackbody}
\usepackage{palapi}

\palCreateFromPal{BlackbodyTransformed}[
  extract = {1, 3, 6, 9},
  shift   = 1,
  reverse
]{Blackbody}
```

Tip.

`\palCreateFromPal{<new-name>}{<existing-name>}` builds a copy of an existing palette.

d. Retrieving the internal definition of a palette

The internally stored definition of a palette named `MyPal`, for example, is `\g_palette_MyPal_seq` which is a LaTeX3 variable (keep in mind the pattern `\g_palette_PaletteName_seq`).

Note.

Variables of type `\g_palette_PaletteName_seq` are not used internally to retrieve the colors themselves; they are only there for technical reasons related to the development process of new palettes via LaTeX.

4. Lua

Note.

Initially, the `@prism` project was created to provide ready-to-use color palettes for `luadraw`, a package that greatly facilitates the creation of high-quality 2D and 3D plots using LuaLaTeX and `TikZ`.

a. Basic use

Note.

All formats are provided: modular (each palette is in a dedicated file) and monolithic (files provide all the palettes).

The `Lua` palette variables are named using the prefix `pal`. They are arrays of arrays of three floats (making it straightforward to use a color from a palette). For example, the definition of `palGistHeat` looks like the following partial code.

```
palGistHeat = {
    {0.0, 0.0, 0.0},
    {0.105882, 0.0, 0.0},
    {0.211764, 0.0, 0.0},
    -- Other RBG colors.
}
```

b. Creating palettes from existing ones

The `palCreateFromPal` function provides options to build new palettes by transforming existing ones. The following example shows how to do this (all options are used).

```
palBlackbodyTransformed = palCreateFromPal(
    palBlackbody,
    {
        extract = {2, 5, 8, 9},
        shift   = 1,
        reverse = true
    }
)
```

VI. Contribute via Git

Caution.

Never use the `main` *branch, which is for freezing the latest stable versions of each projects in the mono repository* `https://github.com/projetmbc/for-writing`.

1. Complete the translations

Important.

Although we're going to explain how to translate the documentation, it doesn't seem relevant to do so, as English should suffice these days.

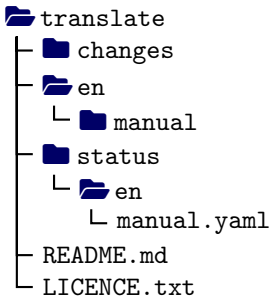


Figure 1: Simplified view of the translation folder

The translations are roughly organized as in figure 1 where just the important folders for the translations have been "*opened*".[5] **A little further down, the section VI-1-e explains how to add new translations**.

a. The en folder

This folder, managed by the author of `@prism`, contains files easy to translate even if you're not a coder.

b. The changes folder

This folder is a communication tool where important changes are indicated without dwelling on minor modifications specific to one or more translations.

c. The status folder

This folder is used to keep track of translations from the project's point of view. Everything is done via well-commented `YAML` files, readable by a non-coder.

d. The README.md and LICENCE.txt files

The `LICENCE.txt` file is aptly named, while the `README.md` file takes up in English the important points of what is said in this section about new translations.

e. New translations

Note.

The folder `manual` *is reserved for documentation. It contains* `TEX` *files that can be compiled directly for real-time validation of translations.*

Warning.

Only start from the `en` *folder, as it's the responsibility of the* `@prism` *author.*

Let's say you want to add support for Italian.[6] To do this, you must use `Git` as follows.

1. Via `https://github.com/projetmbc/for-writing/tree/aprism/@prism`, recover the entire project folder. Do not use the `main` branch, which is used to freeze the latest stable versions of all the projects in the mono repository `https://github.com/projetmbc/for-writing`.
2. In the `@prism/contrib/translate` folder, create an `it` copy of the `en` folder, where `it` is the short name of the language documented in the page "*IIETF language tag*" from `Wikipedia`.
3. Once the translation is complete in the `it` folder, share it via `https://github.com/projetmbc/for-writing/tree/aprism/@prism` using a classic `git push`.

2. Improving the source code

Participation as a coder is made via the repository `https://github.com/projetmbc/for-writing/tree/aprism/@prism` corresponding to the `@prism` development branch. Here is what you can do, details can be found in the file `https://github.com/projetmbc/for-writing/blob/aprism/@prism/contrib/products/README.md`.

1. Create new palettes within an existing implementation. No coding skills required.
2. Propose a new implementation in your favorite programming language.
3. Combine both approaches.

[5] This was the organization on October 26, 2025.
[6] As mentioned above, there is no real need for the `doc` folder.

VII. History

1.3.0
2026-01-01

Break.

- luadraw product becomes Lua product.
- Lua product: the dict-like variable palNames has been removed. Therefore, the responsibility of adding this specific variable will lie with luadraw.

New.

- New CSS and LaTeX products have been coded.
- The Palettable palettes have been added.
- Three semantic palettes TdocBW, TdocCol and TdocDark have been created to be used by the tutodoc class (LaTeX creation process used).
- Added an online demo of @prism (the CSS product showcase is used).

Update.

- Manual.
  - Added lists of renamed and removed palettes.
  - Added two appendixes, one with all palettes grouped by user-friendly categories, and another showing similar palettes.
  - Added explanations about the utility of palettes for non-designers.
  - Added explanations about the minimal common optional sets of extra features implemented.
- Better palette references.
- Better showcases.
  - Several category PDF files are proposed (one PDF per category and theme).
  - 2D chart and Delaunay triangulation added.

Fix.

- Lua product: the use-std.tex and use-dark.tex codes have been fixed.

1.2.1
2025-11-09

Fix.

- Equal palettes: the floating point equality uses now a correct tolerance.

Break.

- Palettes: the extra Greys has been removed (it is equal to Grays).

New.

- Similar palettes: two PDF files show similar palettes in standard and black modes (semi-automated process used).

Update.

- luadraw product: the associative array palNames has been added for compatibility reasons with the luadraw package.
- BlindFish palette: the last color variation has been made smoother (luadraw process used).

1.2.0
2025-10-29

Break.

- Palettes: all final palettes now consist of 10 colors.
- luadraw products: the getPal dictionary array has been converted into a function accepting string palette names (with or without pal prefix). See below.

New.

- Palettes.
  - Added Lemon and ShiftRainbow palettes (luadraw creation process used).
  - Added 37 palettes from the Scientific Coulour Maps project.
- luadraw product: accessing a palette and creating new ones can be made using the getPal function which has an optional argument options (dict-like array) with the following keys and their values.
  - extract: a list of non-zero integers used to extract specific colors from the palette (the order is preserved).
  - reverse: a boolean value indicating whether to reverse the palette color order (false by default).
  - shift: an integer value for applying a circular color shift to the palette.
- Documentations
  - Added English PDF manual.
  - Showcase: two PDF files demonstrate the use of each palette (white and dark modes).

1.1.0
2025-10-14

Break.

- Duplicate palettes and those that are reverse of others are ignored (strict equalities only).

New.

- New palettes added: `BurningGrass`, `GeoRainbow` and `PastelRainbow` (`luadraw` creation process used).
- The `luadraw` palette product has a new dictionary like variable `getPal` to access a palette using its name (as a string variable).

Update.

- Palette contributions: in the mandatory `extend.py` file, the `build_code` function must work with the dictionary of all the palettes, and manage a credit to the `@prism` project.

1.0.0
2025-10-11

First public version of the project.

Appendix 1 – All palette names

Warning.

The palette names in this appendix are standard, but some `@prism` *implementations add a specific prefix, and the very last new palettes are marked by ★.*

A

Accent
Acton
Afmhot
AgGrnYl
AgSunset
Algae
Alphabet
Amp
Antique
Aquatic1
Aquatic2
Aquatic3
ArmyRose
Autumn

B

Balance ★
Bam
BamO
Bamako
Batlow
BatlowK
BatlowW
Berlin
Bilbao
Binary
Blackbody
BlindFish
BluGrn
BluYl
BlueDarkOrange12
BlueDarkOrange18
BlueDarkRed12
BlueDarkRed18
BlueGray
BlueGreen
BlueOrange8
BlueOrange10
BlueOrange12
BlueOrangeRed
BlueRed
BlueRedPLY
Bluegrey ★
Blues
Blues7
Blues10
BluesCP ★
Bold
Bone
BrBG
Brg
Broc
BrocO
BrownBlue10
BrownBlue12
BrwnYl
BuGn
BuPu
Buda
Bukavu
Burg
BurgYl
BurningGrass
Bwr

C

CMRmap
Cavalcanti
Chevalier
Cividis
Classic ★
ColorBlind
Cool
CoolCP ★
Coolwarm
Copper
Cork
CorkO
Cube1
CubeYF
Cubehelix
Cubehelix1
Cubehelix2
Cubehelix3
Curl

D

Darjeeling1
Darjeeling2
Darjeeling3
Darjeeling4
Dark2
Dark24
DarkMint
Davos
Deep
Delta
Dense
Devon

E

Earth
EarthCP ★
Electric
Emrld

F

Fall
FantasticFox1
FantasticFox2
Fes
Fire ★
Flag

G

G10
GasFlame
GeoRainbow
Geyser
GistEarth
GistGray ★
GistHeat
GistNcar
GistRainbow
GistStern
Glasgow
GnBu
Gnuplot
Gnuplot2
GrandBudapest1
GrandBudapest2

GrandBudapest3
GrandBudapest4
GrandBudapest5
Gray ★
GrayC
GreenMagenta
GreenOrange
Greengrey ★
Greens
GreensCP ★
Greys ★
GreysCP ★

H

Haline
Hawaii
Heart ★
Hot
HotPLY ★
Hsv

I

Ice
Imola
Inferno
IsleOfDogs1
IsleOfDogs2
IsleOfDogs3

J

Jet
JetPLY ★
JimSpecial

L

Lajolla
Lapaz
Lemon
Light24
LinearL
Lipari
Lisbon

M

Magenta
Magma
Managua
Margot1
Margot2
Margot3
Matter
Mendl
MidGray ★
Mint
Moonrise1
Moonrise2
Moonrise3
Moonrise4
Moonrise5
Moonrise6
Moonrise7

N

Navia
NaviaW
NipySpectral
Nuuk

O

Ocean
OkabeIto ★
Oleron
OrRd
OrYel
Orangeblue ★
Orangegrey ★
Oranges
OrangesCP ★
Oslo
Oxy

P

PRGn
Paired
Pastel
Pastel1
Pastel2
PastelRainbow
Peach
PerceptualRainbow
Petroff6 ★
Petroff8 ★
Petroff10 ★
Phase
PiYG
Picnic
Pink
PinkYl
Pinkgreen ★
Pinkgrey ★
Pinks ★
Plasma
Plotly
Portland
Prism
PrismCC
PuBu
PuBuGn
PuOr
PuRd
Purp
PurpOr
Purple
PurpleGray
Purplegrey ★
Purples
PurplesCP ★

R

Rainbow
RainbowCP ★
RainbowPLY
RdBu
RdGy
RdPu
RdYlBu
RdYlGn
Red
RedOr
RedYellowBlue
Redgrey ★
Reds
RedsCP ★
Redteal ★
Roma
RomaO
Royal1
Royal2
Royal3

S

Safe
Seismic
Set1
Set2
Set3
ShiftRainbow
Sinebow ★
Solar
Spectral
Speed
Spring
Summer
Sunset
SunsetDark

T

T10
Tab10
Tab20
Tab20b
Tab20c
TableauLight
TableauMedium
TdocBW
TdocCol
TdocDark
Teal
TealGrn
TealRose
Tealgrey ★
Teals ★
Tempo
Temps
Terrain
Thermal
Tofino
Tokyo
TrafficLight
Tropic
Turbid
Turbo
Turku
Twilight
TwilightShifted

V

Vanimo
Vik
VikO
Viridis
Vivid

W

Warm ★
Water ★
Wind ★
Winter
Wistia

Y

Yellowgrey ★
Yellowpurple ★
Yellows ★
YlGn
YlGnBu
YlOrBr
YlOrRd
YlOrRdCBW ★

Z

Zissou

Appendix 2 – Palettes by category

Warning.

The palette names in this appendix are standard, but some `@prism` *implementations add a specific prefix.*

Note (Qualitative and semantic palettes).

The threshold of 40 colors has been established for qualitative and semantic palettes. The following lines outline the rationale for this limit.

- *In qualitative color palettes, each color represents a single data point or a group of data. Experience shows that about ten colors are more than enough.*
- *Semantic palettes associate a color with a specific context, such as warning levels (see the three frames in this document). Choosing 40 colors allows for 10 different contexts, each with 4 sub-categories, for instance, a website where each context has specific states for 'visited', 'unvisited', or 'hovered' content.*

Important.

Categories were generated semi-automatically, followed by manual selection to obta in relevant choices.

Colorblind palettes

Colorblind palettes provide high-contrast hues specifically optimized for users with color vision deficiency.

Acton

Bam

BamO

Bamako

Batlow

BatlowK

BatlowW

Berlin

Bilbao

Broc

BrocO

Buda

Bukavu

Cork

CorkO

Cubehelix

Davos

Devon

Fes

Glasgow

GrayC

Hawaii

Imola

Lajolla

Lapaz

Lipari

Lisbon

Managua

Navia

NaviaW

Nuuk

OkabeIto

Oleron

Oslo

Petroff6

Petroff8

Petroff10

Roma

RomaO

Safe

Tofino

Tokyo

Turku

Vanimo

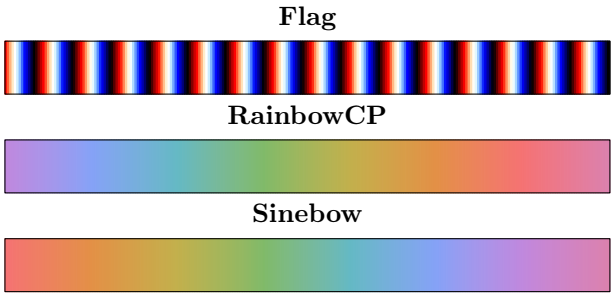
Vik

VikO

Wistia

Cyclic palettes

Cyclic palettes create seamless loops for periodic data, such as angles, seasons, or 24-hour cycles.



Dark palettes

Dark palettes offer high-contrast schemes specifically optimized for terminal interfaces and dark-mode backgrounds.

Autumn

Bwr

Cool

GistRainbow

Hsv

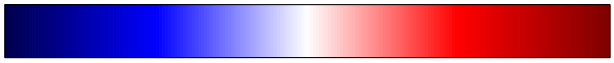
Jet

Lemon

Prism

Rainbow

Seismic

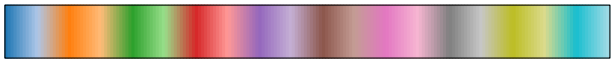


Spring

Summer

Tab10

Tab20



Tab20b

Tab20c

TdocDark

Terrain

Viridis

Winter

YlOrRd

Divergent palettes

Divergent palettes highlight deviations from one or more midpoints toward contrasting extremes.

ArmyRose

Balance

BlindFish

BlueDarkOrange12

BlueDarkOrange18

BlueDarkRed12

BlueDarkRed18

BlueGray

BlueGreen

BlueOrange8

BlueOrange10

BlueOrange12

BlueOrangeRed

Bluegrey

BrBG

BrownBlue10

BrownBlue12

BurningGrass

Bwr

Coolwarm

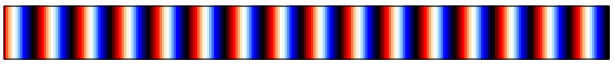
Curl

Delta

Earth

Fall

Flag



Geyser

GistNcar

GistRainbow

GistStern

GreenMagenta

Greengrey

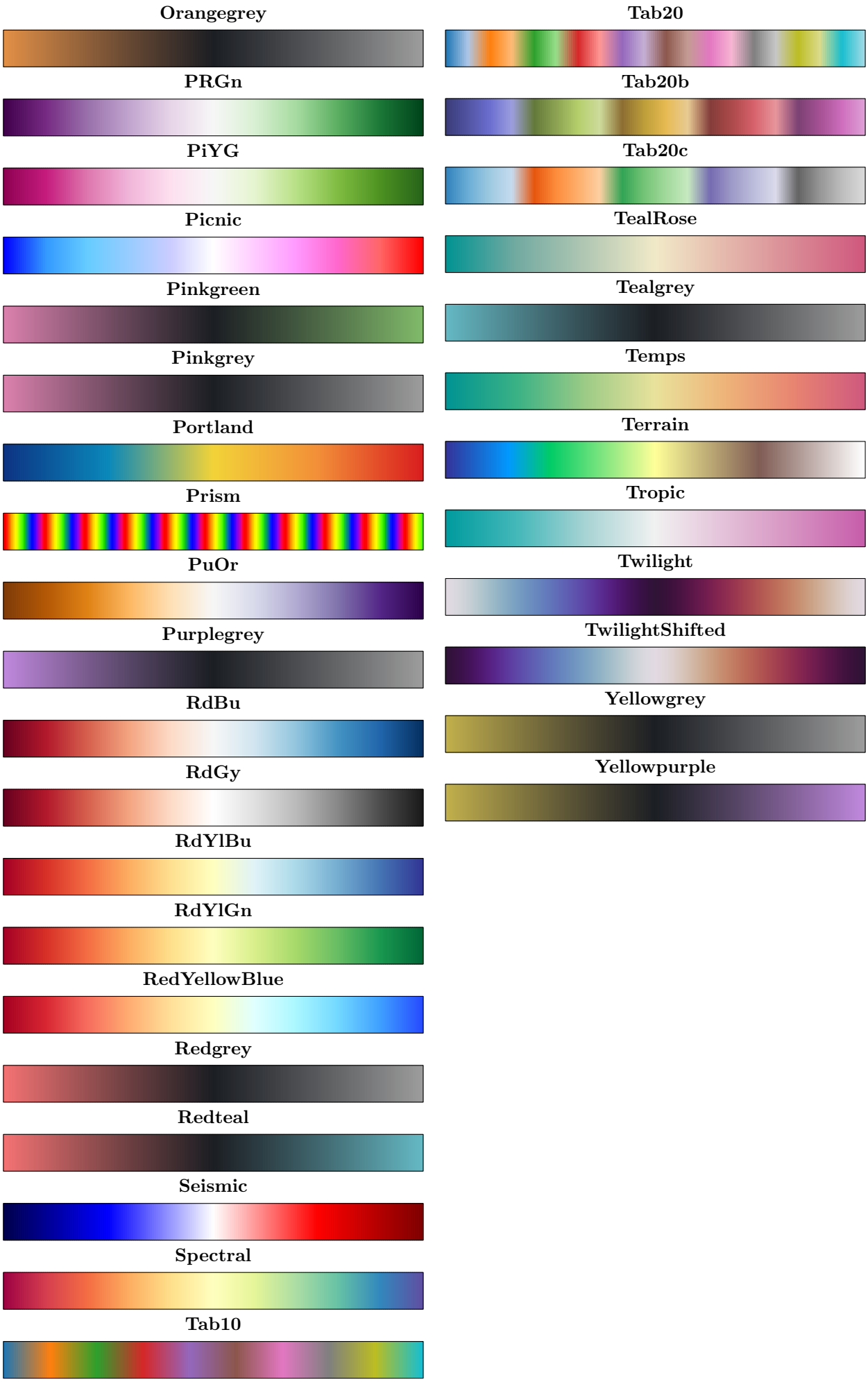
Hsv

Jet

NipySpectral

Ocean

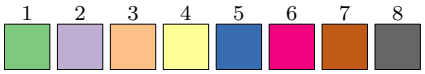
Orangeblue


Orangegrey
PRGn
PiYG
Picnic
Pinkgreen
Pinkgrey
Portland
Prism
PuOr
Purplegrey
RdBu
RdGy
RdYlBu
RdYlGn
RedYellowBlue
Redgrey
Redteal
Seismic
Spectral
Tab10
Tab20
Tab20b
Tab20c
TealRose
Tealgrey
Temps
Terrain
Tropic
Twilight
TwilightShifted
Yellowgrey
Yellowpurple

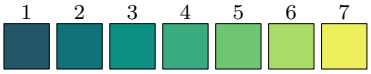
Qualitative palettes

Qualitative palettes use distinct hues to separate categorical data without implying any order or hierarchy.

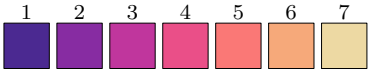
Accent



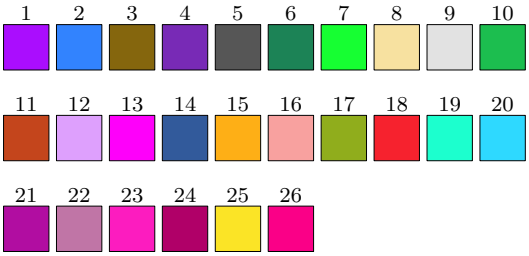
AgGrnYl



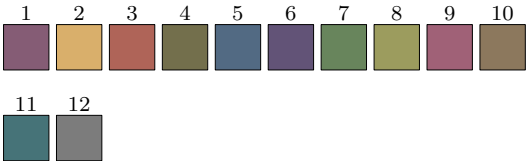
AgSunset



Alphabet

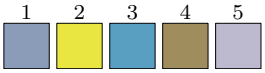


Antique

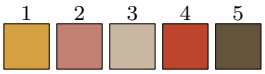


Aquatic1

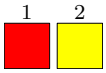
Aquatic2



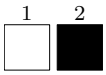
Aquatic3



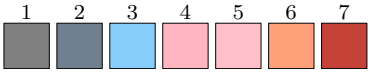
Autumn



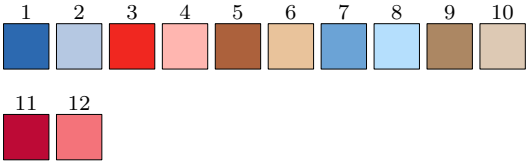
Binary



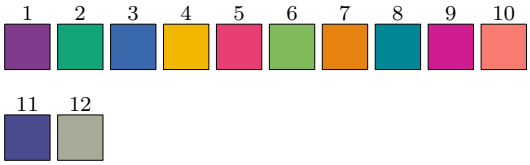
BlindFish



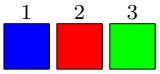
BlueRed



Bold

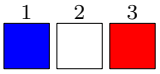


Brg

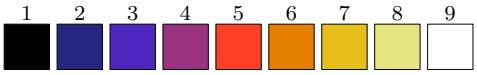


BurningGrass

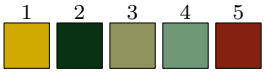
Bwr



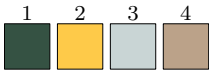
CMRmap



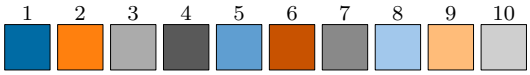
Cavalcanti



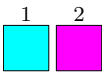
Chevalier



ColorBlind

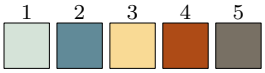


Cool

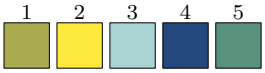


Darjeeling1

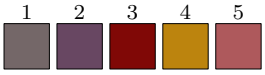
Darjeeling2



Darjeeling3

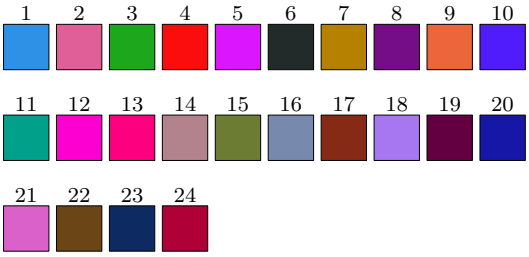


Darjeeling4

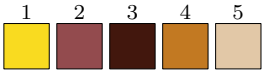


Dark2

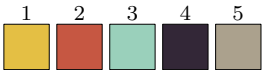
Dark24



FantasticFox1

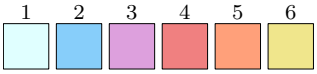


FantasticFox2



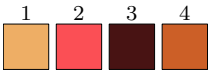
G10

GasFlame

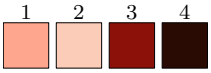


GeoRainbow

GrandBudapest1

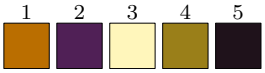


GrandBudapest2

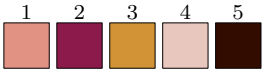


GrandBudapest3

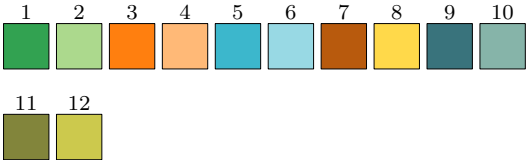
GrandBudapest4



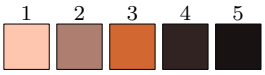
GrandBudapest5



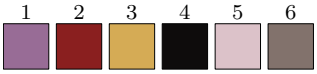
GreenOrange



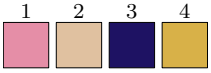
IsleOfDogs1



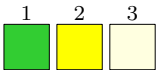
IsleOfDogs2



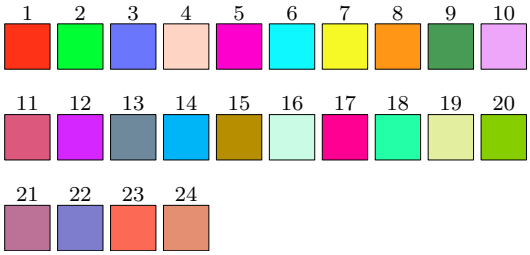
IsleOfDogs3



Lemon

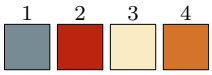


Light24

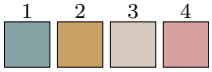


Margot1

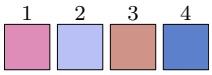
Margot2



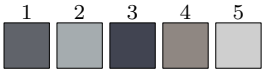
Margot3



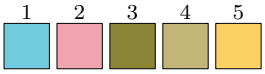
Mendl



MidGray

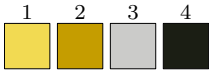


Moonrise1

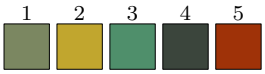


Moonrise2

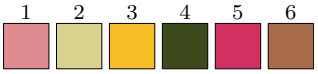
Moonrise3



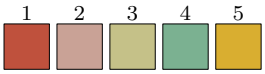
Moonrise4



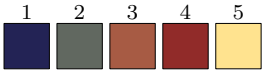
Moonrise5



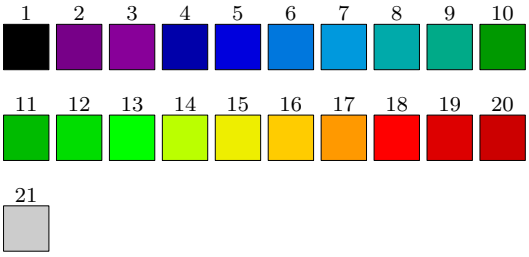
Moonrise6



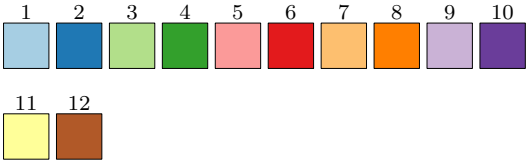
Moonrise7



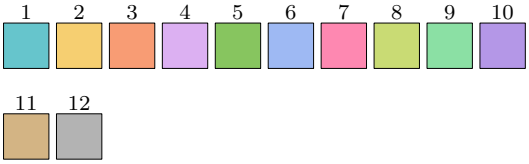
NipySpectral



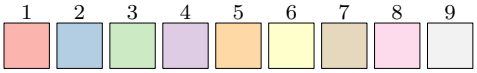
Paired



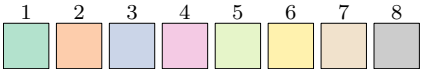
Pastel



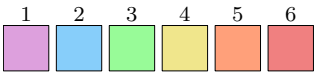
Pastel1



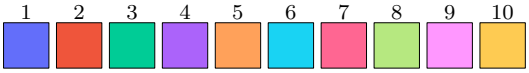
Pastel2



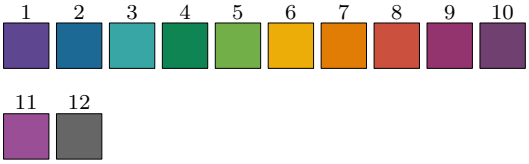
PastelRainbow



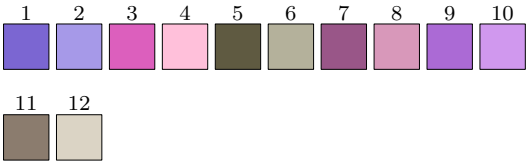
Plotly



PrismCC

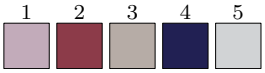


PurpleGray



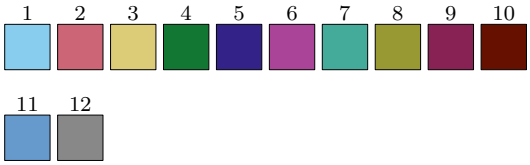
Royal1

Royal2

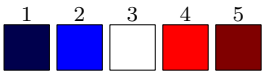


Royal3

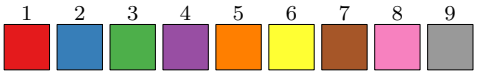
Safe



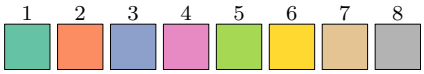
Seismic



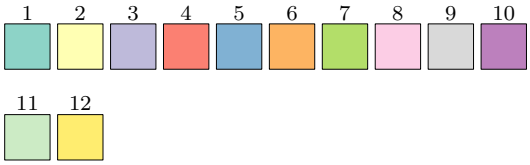
Set1



Set2

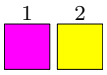


Set3

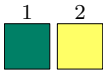


ShiftRainbow

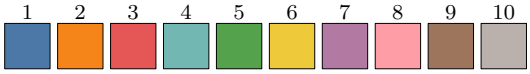
Spring



Summer

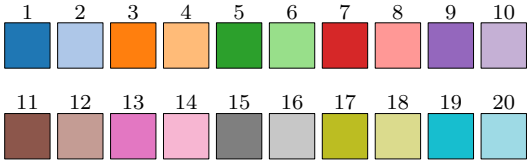


T10

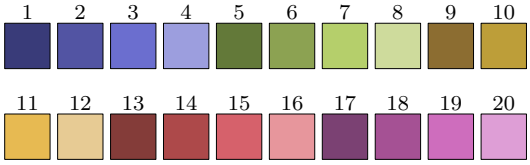


Tab10

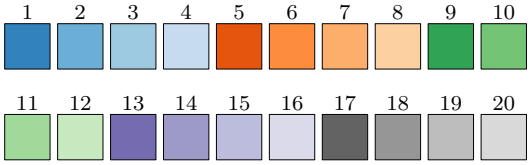
Tab20



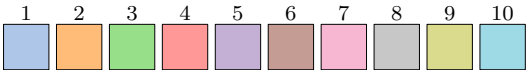
Tab20b



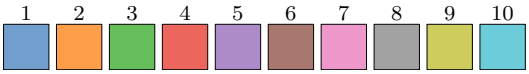
Tab20c



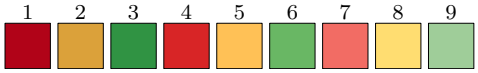
TableauLight



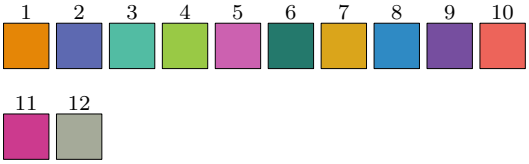
TableauMedium



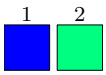
TrafficLight



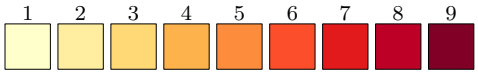
Vivid



Winter



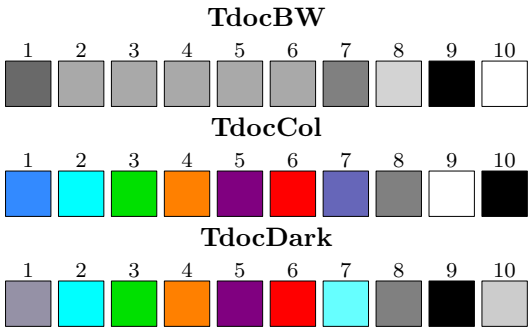
YlOrRd



Zissou

Semantic palettes

Semantic palettes assign colors to categories — such as UI states, system status, or pedagogical elements — to convey immediate meaning.



Sequential palettes

Sequential palettes represent ordered data using a single or multi-hue gradient with increasing lightness.

Afmhot

Algae

Amp

Autumn

Binary

Blackbody

BlindFish

BluGrn

BluYl

BlueRedPLY

Blues

Blues7

Blues10

BluesCP

Bone

Brg

BrwnYl

BuGn

BuPu

Burg

BurgYl

BurningGrass

Bwr

CMRmap

Cividis

Classic

Cool

CoolCP

Copper

Cube1

CubeYF

Cubehelix

Cubehelix1

Cubehelix2

Cubehelix3

DarkMint

Deep

Dense

EarthCP

Electric

Emrld

Fire

GasFlame

GeoRainbow

GistEarth

GistGray

GistHeat

GistNcar

GistRainbow

GistStern

GnBu

Gnuplot

Gnuplot2

Gray

Greens

GreensCP

Greys

GreysCP

Haline

Heart

Hot

HotPLY

Hsv

Ice

Inferno

Jet

JetPLY

JimSpecial

Lemon

LinearL

Magenta

Magma

Matter

Mint

NipySpectral

Ocean

OrRd
OrYel
Oranges
OrangesCP
Oxy
PastelRainbow
Peach
PerceptualRainbow
Phase
Pink
PinkYl
Pinks
Plasma
PuBu
PuBuGn
PuRd
Purp
PurpOr
Purple
Purples
PurplesCP
Rainbow
RainbowPLY
RdPu
Red
RedOr
Reds
RedsCP
Seismic
ShiftRainbow
Solar
Speed
Spring
Summer
Sunset
SunsetDark
Teal
TealGrn
Teals
Tempo

Terrain

Thermal

Turbid

Turbo

Twilight

TwilightShifted

Viridis

Warm

Water

Wind

Yellows

YlGn

YlGnBu

YlOrBr

YlOrRd

YlOrRdCBW